Universidad del Valle de Guatemala

CC3045 – Deep Learning

Laboratorio 9

Autoencoder variacional y modelo de difusión

Fashion-MNIST

Integrantes: Milton Polanco
Olivier Viau

Semilla: 42

Fecha: 10 de octubre de 2026

Entrega final

Índice

1. Resumen **1**

2. Configuración experimental **1**

3. Task 1 – Autoencoder variacional **2**

3.1. Task 1.1 – Datos y configuración 2

3.2. Task 1.2 – Reparametrización, KL y entrenamiento 2

3.3. Task 1.3 – Representación latente y generación 3

3.4. Task 1.4 – Análisis cuantitativo 4

4. Task 2 – Proceso forward de difusión **7**

4.1. Task 2.1 – Calendario lineal y forma cerrada 7

4.2. Task 2.2 – Verificación empírica 7

4.3. Task 2.3 – Evolución visual y SNR 8

4.4. Task 2.4 – Calendario coseno truncado 8

4.5. Task 2.5 – Análisis del calendario 9

5. Task 3 – Difusión condicional **12**

5.1. Task 3.1 – U-Net y condicionamiento 12

5.2. Task 3.2 – Entrenamiento y pérdida por intervalo 12

5.3. Task 3.3 – Muestreo ancestral 13

5.4. Task 3.4 – Guía sin clasificador 14

5.5. Task 3.5 – Análisis de la guía 14

6. Task 4 – Comparaciones y pass@k **17**

6.1. Task 4.1 – VAE frente a difusión 17

6.2. Task 4.2 – Estimador pass@k 17

7. Conclusiones **18**

Uso de herramientas de IA **19**

1. Resumen

Se implementaron desde cero un autoencoder variacional (VAE) y un modelo de difusión condicional sobre Fashion-MNIST. El VAE se entrenó con tres valores de β y un espacio latente bidimensional. Para difusión se construyó el proceso forward, se verificó su forma cerrada y se entrenó una U-Net pequeña con condicionamiento temporal y de clase. Todas las métricas proceden de la ejecución del notebook con semilla 42.

Los resultados muestran el compromiso esperado. En el VAE, aumentar β reduce la información del código latente y empeora la reconstrucción. En difusión, una guía más fuerte eleva la fidelidad de 0.838 a 0.980, pero reduce la diversidad de 14.927 a 12.081. El modelo de difusión genera imágenes más nítidas que el VAE (0.0736 frente a 0.0499), aunque necesita 1000 evaluaciones de red y fue aproximadamente 14471 veces más lento en la medición realizada. Finalmente, el estimador insesgado de pass@k confirma que los reintentos con verificación pueden aumentar de forma importante la probabilidad de resolver un problema, pero no compensan la ausencia total de soluciones correctas.

2. Configuración experimental

Fashion-MNIST contiene 60000 imágenes de entrenamiento y 10000 de prueba, todas de 28×28 píxeles en escala de grises y distribuidas en diez clases. Se reservaron los últimos 5000 elementos del conjunto de entrenamiento para validación, por lo que se usaron 55000 imágenes para ajustar los modelos. Los píxeles se escalaron a $[0, 1]$ para el VAE y a $[-1, 1]$ para difusión.

La semilla 42 se fijó en Python, NumPy y PyTorch. Los tres VAE se inicializaron nuevamente con la misma semilla para que la comparación entre valores de β partiera de pesos equivalentes. El código utiliza tensores de PyTorch y no emplea bibliotecas de difusión, calendarios prefabricados, muestreadores externos ni funciones de divergencia KL ya implementadas.

Cuadro 1: Configuración principal de los experimentos.

Parámetro	VAE	Difusión
Tamaño de lote	128	128
Épocas	20	30
Tasa de aprendizaje	10^{-3}	2×10^{-4}
Dimensión latente	2	–
β del VAE	0.1, 1, 10	–
Pasos de difusión T	–	1000
Probabilidad no condicional	–	0.1

3. Task 1 – Autoencoder variacional

3.1. Task 1.1 – Datos y configuración

Después de normalizar el conjunto a $[0, 1]$, los tensores de entrenamiento y validación tuvieron formas $(55000, 784)$ y $(5000, 784)$. La arquitectura utilizada fue completamente conectada: el encoder siguió $784 \to 512 \to 256$ y produjo μ y $\log \sigma^2$ de dimensión 2; el decoder siguió $2 \to 256 \to 512 \to 784$ y terminó con una sigmoide.

3.2. Task 1.2 – Reparametrización, KL y entrenamiento

El muestreo latente se implementó mediante

$$z = \mu + \sigma \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I), \qquad \sigma = \exp\left(\frac{1}{2} \log \sigma^2\right). \tag{1}$$

Para una posterior diagonal $q(z \mid x) = \mathcal{N}(\mu, \mathrm{diag}(\sigma^2))$ y el prior $p(z) = \mathcal{N}(0, I)$, la divergencia por imagen fue

$$D_{\mathrm{KL}}(q \| p) = -\frac{1}{2} \sum_{j=1}^{2} \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right). \tag{2}$$

La función objetivo se calculó por imagen, sumando el error cuadrático en los 784 píxeles:

$$\mathcal{L}_\beta(x) = \|x - \hat{x}\|_2^2 + \beta D_{\mathrm{KL}}(q(z \mid x) \| p(z)). \tag{3}$$

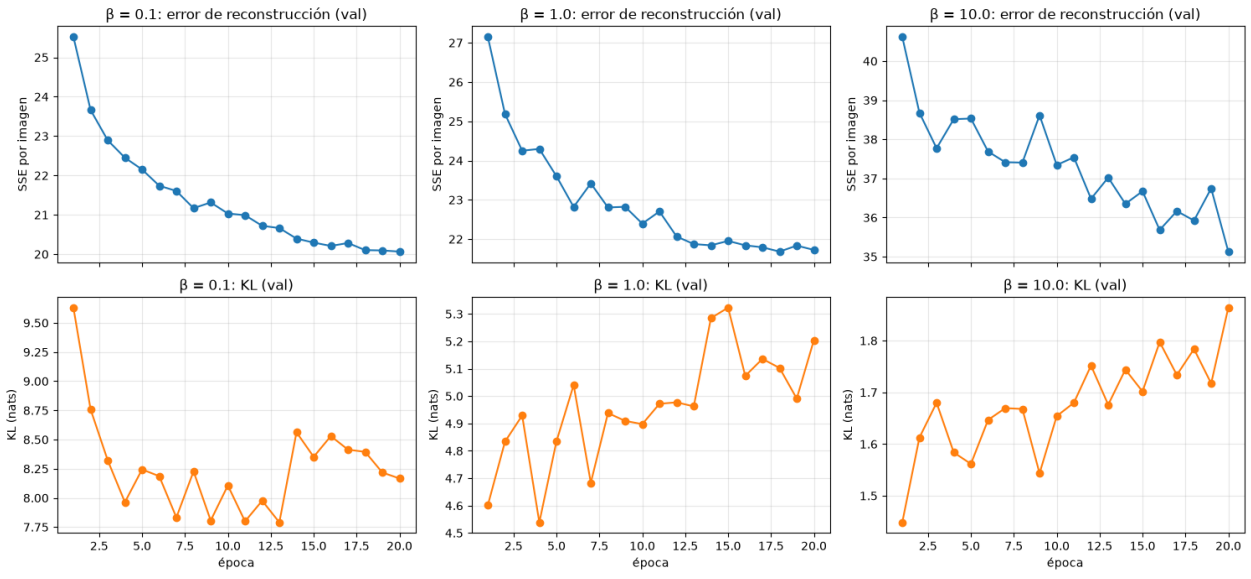


Figura 1: Error de reconstrucción y KL de validación durante el entrenamiento de los tres VAE.

La Figura 1 muestra que un β mayor obliga al posterior a acercarse más al prior, reduciendo la KL, pero eleva el error de reconstrucción. Con $\beta = 0.1$ el modelo conserva más información específica de cada imagen; con $\beta = 10$ la regularización domina la optimización.

3.3. Task 1.3 – Representación latente y generación

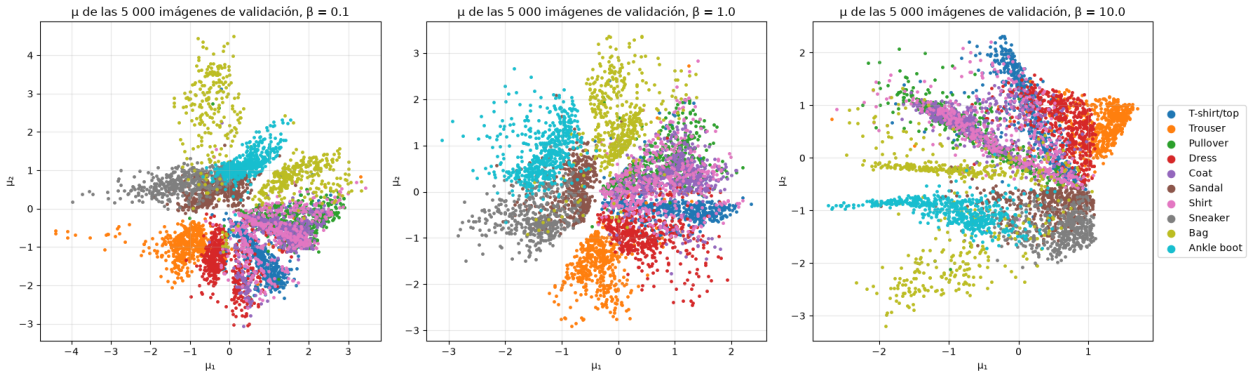


Figura 2: Medias latentes de las 5000 imágenes de validación, coloreadas por clase.

En la Figura 2, $\beta = 0.1$ ocupa una región amplia y separa mejor varios grupos. Al aumentar β, las medias se concentran alrededor del origen y las clases visualmente similares se superponen. El efecto es especialmente claro con $\beta = 10$.

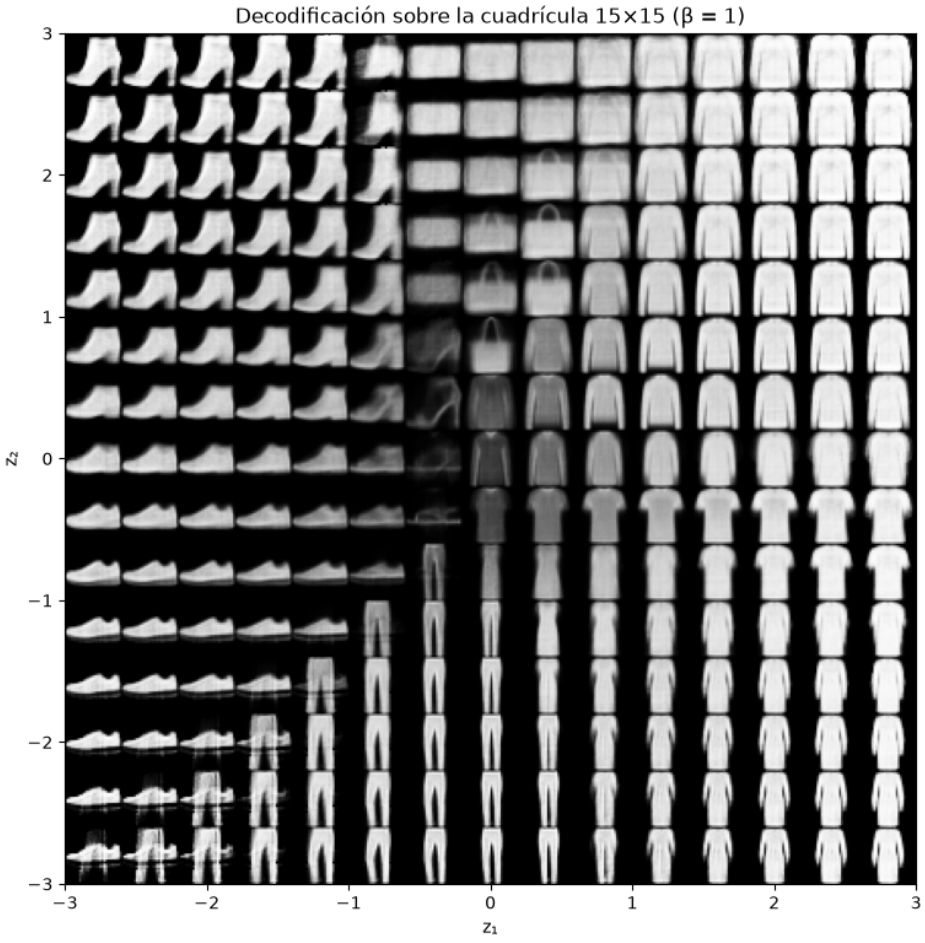


Figura 3: Cuadrícula regular de 15×15 puntos decodificados para $\beta = 1$ y $z \in [-3, 3]^2$.

La cuadrícula de la Figura 3 permite observar transiciones continuas entre tipos de prenda. Alrededor de $z_1 \approx -0.5$ y $z_2 \approx 0$ aparecen imágenes oscuras o ambiguas. En regiones extremas también hay mezclas poco reconocibles porque el prior asigna poca probabilidad y hay pocos códigos de entrenamiento cercanos.

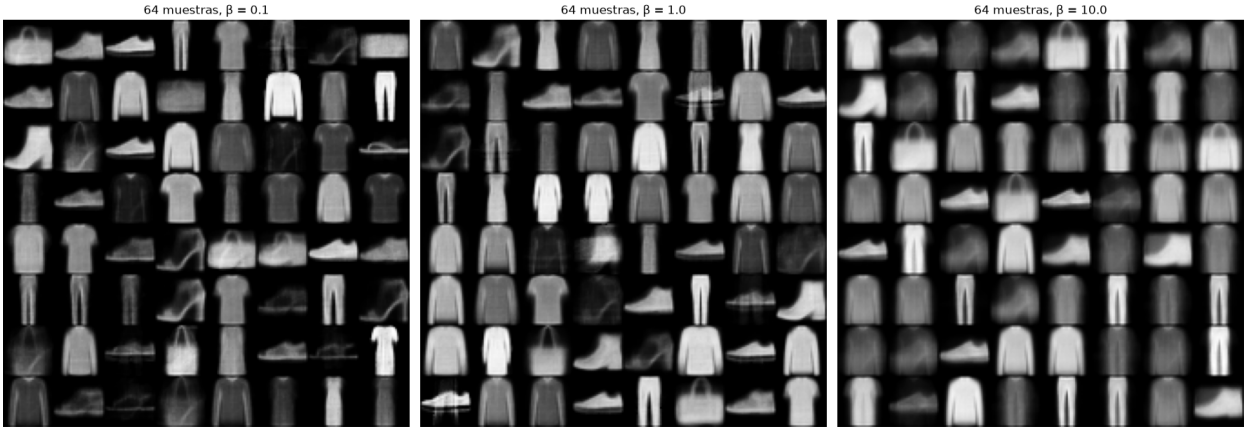


Figura 4: Sesenta y cuatro muestras aleatorias de cada VAE, obtenidas con $z \sim \mathcal{N}(0, I)$.

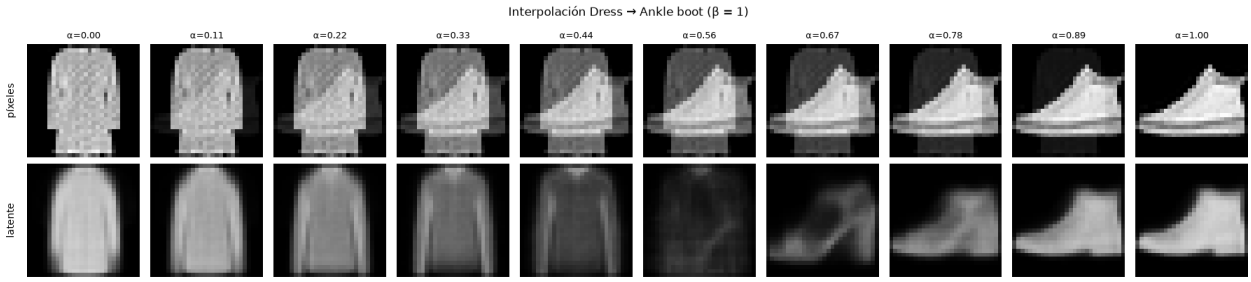


Figura 5: Interpolación entre dos clases distintas en píxeles y en el espacio latente del VAE con $\beta = 1$.

En el punto medio de la interpolación directa en píxeles se superponen las siluetas del vestido y de la bota. El resultado conserva partes de ambas imágenes, pero no parece una prenda real. En el punto medio latente, el decoder produce una sola forma vertical, borrosa y ambigua. La diferencia ocurre porque la primera operación promedia intensidades, mientras que la segunda recorre la representación aprendida y decodifica un punto intermedio.

3.4. Task 1.4 – Análisis cuantitativo

(a) Reconstrucción, KL y capacidad del código

Cuadro 2: Resultados finales de validación de los VAE.

β	Reconstrucción (SSE/img)	KL (nats)	KL (bits)
0.1	20.065	8.167	11.782
1.0	21.723	5.204	7.508
10.0	35.130	1.864	2.689

La KL media de $\beta = 10$ equivale a 2.689 bits, menos que $\log_2(10) = 3.322$ bits. Además,

$$I(y; z) \leq I(x; z) \leq \mathbb{E}_x \left[D_{\mathrm{KL}}(q(z \mid x) \| p(z)) \right]. \tag{4}$$

Por tanto, en promedio el código no dispone de información suficiente para identificar sin error una entre diez clases equiprobables. El diagrama latente coincide con esta cota: todavía se distinguen grupos generales como pantalones, calzado y bolsos, pero camiseta, pullover, abrigo, vestido y camisa presentan una superposición marcada.

(b) Verificación Monte Carlo de la KL

Para la primera imagen de validación con $\beta = 1$ se obtuvo $\mu = (1.541, 0.263)$ y $\sigma = (0.171, 0.092)$. La KL Monte Carlo se estimó con 10^5 muestras mediante el promedio de $\log q(z) - \log p(z)$.

Cuadro 3: Comparación de la KL cerrada y la estimación Monte Carlo.

Cantidad	Valor
KL cerrada	4.393141 nats
KL Monte Carlo	4.398244 nats
Diferencia absoluta	0.005103 nats
Error relativo	0.001161 (0.116 %)
Error estándar Monte Carlo	3.193×10^{-3}

La diferencia observada equivale a 1.60 errores estándar. Es compatible con la variabilidad esperada de una estimación de este tamaño y no sugiere un error de signo ni un factor constante en la expresión cerrada.

(c) Regiones no reconocibles e interpolación

Con dos dimensiones latentes, varias clases comparten regiones cercanas. En sus fronteras, el decoder entrenado con error cuadrático aproxima el promedio de varias prendas compatibles con el mismo código. En los extremos de la cuadrícula, además, el modelo extrapola. Estos dos efectos explican las zonas ambiguas observadas. La regularización KL reduce huecos en el espacio latente, pero no puede evitar la pérdida de detalle producida por comprimir las imágenes a solo dos valores.

(d) Nitidez y varianza del decoder

Cuadro 4: Nitidez media, definida por diferencias horizontales entre píxeles adyacentes.

Origen	Nitidez	Fracción de la nitidez real
Reales (1000)	0.0891	1.00
VAE, $\beta = 0.1$	0.0436	0.49
VAE, $\beta = 1$	0.0509	0.57
VAE, $\beta = 10$	0.0435	0.49

Para un decoder gaussiano con varianza σ_x^2,

$$-\log p(x \mid z) = \frac{\|x - \hat{x}\|^2}{2\sigma_x^2} + \frac{784}{2}\log(2\pi\sigma_x^2). \tag{5}$$

Al sumar la KL y multiplicar el ELBO negativo por $2\sigma_x^2$, queda

$$\|x - \hat{x}\|^2 + 2\sigma_x^2 D_{\mathrm{KL}}. \tag{6}$$

Comparar esta expresión con $\mathcal{L}_\beta$ da $\sigma_x^2 = \beta/2$. El error cuadrático lleva al decoder a producir la media de todas las imágenes compatibles con un código; con un espacio de solo dos dimensiones, esta media suaviza bordes y detalles. Un β grande añade pérdida de información por regularización, mientras que un β pequeño deja huecos entre códigos y permite que algunas muestras del prior caigan en zonas poco observadas.

4. Task 2 – Proceso forward de difusión

4.1. Task 2.1 – Calendario lineal y forma cerrada

Se utilizaron $T = 1000$ pasos, $\beta_1 = 10^{-4}$ y $\beta_T = 0.02$. El calendario lineal define

$$\alpha_t = 1 - \beta_t, \qquad \bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s, \tag{7}$$

y la muestra en un instante arbitrario se obtiene sin simular los pasos anteriores:

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I). \tag{8}$$

El producto final calculado fue $\bar{\alpha}_T = 4.03583 \times 10^{-5}$.

4.2. Task 2.2 – Verificación empírica

Se fijó una imagen x_0, se crearon 5000 copias y se aplicaron 300 pasos independientes de

$$x_s = \sqrt{\alpha_s}\, x_{s-1} + \sqrt{\beta_s}\, \epsilon_s. \tag{9}$$

Cuadro 5: Verificación empírica de la forma cerrada en $t = 300$.

Cantidad	Resultado
$\bar{\alpha}_{300}$	0.396420
Varianza teórica $1 - \bar{\alpha}_{300}$	0.603580
Error máximo absoluto de la media	0.03549
Error estándar esperado de la media	0.01099
Máximo |error|/SE	3.23
Píxeles dentro de 2 SE	95.4 %
Varianza empírica promedio	0.602884

El máximo de 3.23 errores estándar es razonable al observar simultáneamente 784 píxeles. Además, 95.4 % de los píxeles quedó dentro de dos errores estándar, exactamente el porcentaje esperado aproximadamente para una variable normal. La diferencia entre las varianzas empírica y teórica fue 6.96×10^{-4}, equivalente a 1.61 errores estándar del promedio de las 784 varianzas. La simulación es compatible con la forma cerrada.

4.3. Task 2.3 – Evolución visual y SNR

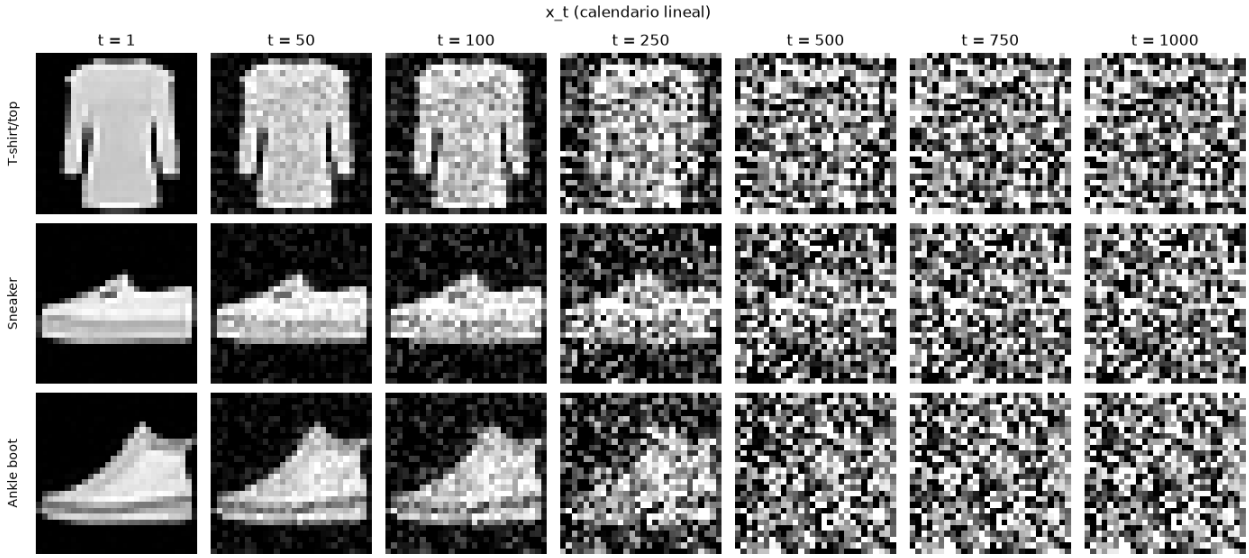


Figura 6: Proceso forward para tres clases y siete instantes del calendario lineal.

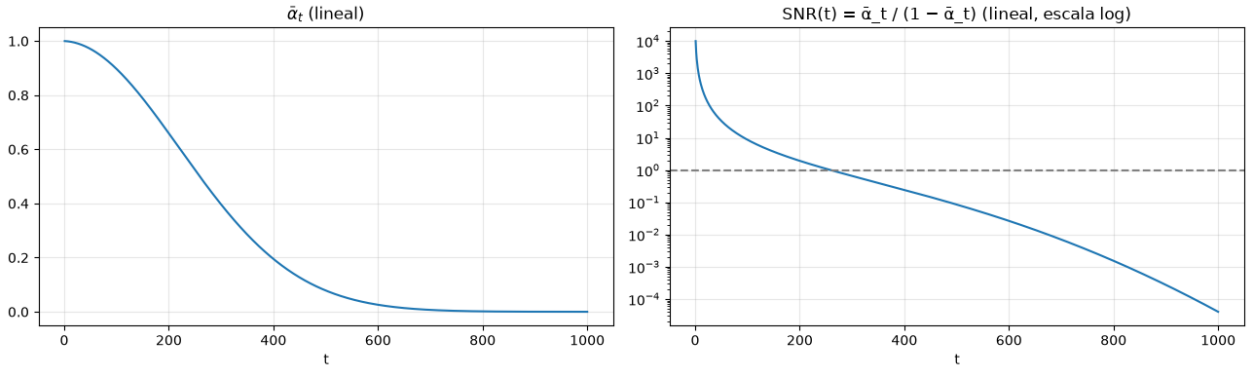


Figura 7: $\bar{\alpha}_t$ y SNR del calendario lineal.

El primer instante con $\mathrm{SNR}(t) < 1$ fue $t = 260$, mientras que $\bar{\alpha}_t < 0.01$ ocurrió por primera vez en $t = 674$. En la Figura 6, hacia $t = 250$ ya se pierde buena parte de la identidad de la prenda y desde $t = 750$ la imagen es ruido casi puro.

4.4. Task 2.4 – Calendario coseno truncado

Como segundo calendario se tomó como base la forma coseno y se modificó mediante un truncamiento temporal:

$$f(u) = \cos^2\left(\frac{u+s}{1+s}\frac{\pi}{2}\right), \qquad \bar{\alpha}_t = \frac{f(rt/T)}{f(0)}, \tag{10}$$

con $s = 0.008$ y $r = 0.993583$. Este valor de r se eligió para imponer $\bar{\alpha}_T = 10^{-4}$ sin recortar manualmente los β_t. Luego,

$$\beta_t = 1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}. \tag{11}$$

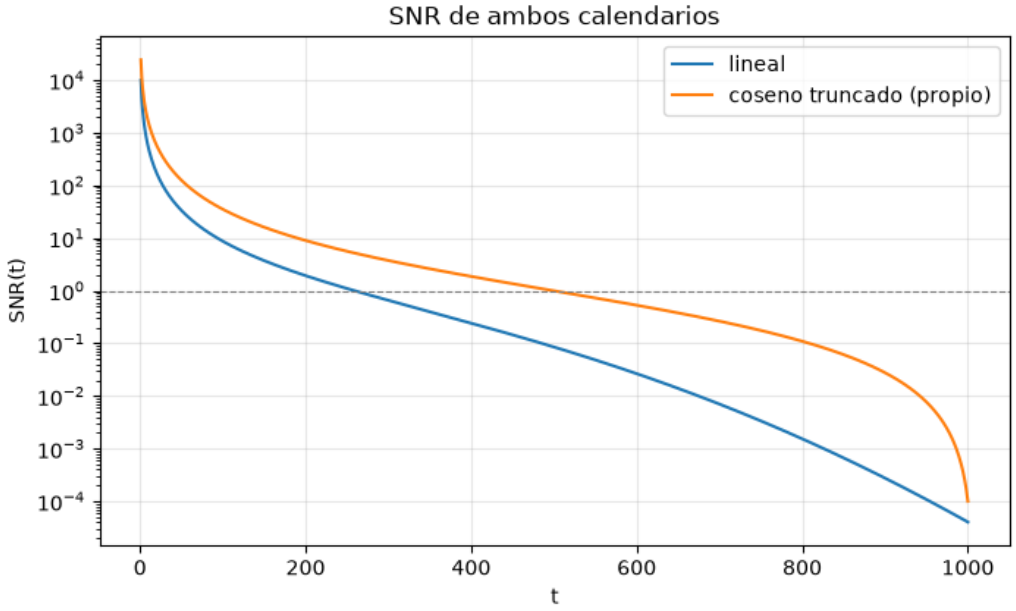


Figura 8: Comparación de la SNR de los calendarios lineal y coseno truncado.

Cuadro 6: Propiedades de los dos calendarios.

Calendario	$\bar{\alpha}_T$	Primer t: SNR < 1	$\beta_{\text{mín}}$	$\beta_{\text{máx}}$
Lineal	4.04×10^{-5}	260	1.00×10^{-4}	0.020
Coseno truncado	1.00×10^{-4}	500	4.10×10^{-5}	0.250

Ambos calendarios cumplen $\bar{\alpha}_T < 10^{-3}$ y $\beta_t \in (0, 1)$. El calendario propuesto conserva una SNR intermedia durante una fracción mayor de la trayectoria.

4.5. Task 2.5 – Análisis del calendario

(a) Aproximación de $\bar{\alpha}_T$

Para el calendario lineal,

$$\log \bar{\alpha}_T = \sum_{t=1}^{T} \log(1 - \beta_t) \tag{12}$$

$$\approx -\sum_{t=1}^{T} \beta_t - \frac{1}{2} \sum_{t=1}^{T} \beta_t^2. \tag{13}$$

La primera suma es aritmética:

$$\sum_{t=1}^{T} \beta_t = \frac{T(\beta_1 + \beta_T)}{2} = \frac{1000(10^{-4} + 0.02)}{2} = 10.05. \tag{14}$$

Para la segunda se aproxima la progresión lineal mediante una integral:

$$\sum_{t=1}^{T} \beta_t^2 \approx T \int_0^1 [\beta_1 + (\beta_T - \beta_1)u]^2 \, du \tag{15}$$

$$= \frac{T(\beta_1^2 + \beta_1\beta_T + \beta_T^2)}{3} \approx 0.1340. \tag{16}$$

Por tanto,

$$\log \bar{\alpha}_T \approx -10.05 - 0.0670 = -10.117, \qquad \bar{\alpha}_T \approx 4.0387 \times 10^{-5}. \tag{17}$$

El producto acumulado del código dio 4.0358×10^{-5}, con error relativo 7.12×10^{-4}. El término de tercer orden omitido es aproximadamente -6.7×10^{-4} en el logaritmo, por lo que la diferencia tiene la magnitud esperada.

(b) Regímenes de SNR

Cuadro 7: Número de pasos en cada régimen de SNR.

Calendario	SNR < 0.01	SNR $\in [0.01, 100]$	SNR > 100
Lineal	326	647	27
Coseno truncado	58	886	56

Cuando SNR $\ll 0.01$, $x_t \approx \epsilon$ y predecir el ruido es casi trivial: la red puede devolver algo muy cercano a su entrada. Estos pasos aportan poca estructura visual al aprendizaje. Cuando SNR $\gg 100$, $x_t \approx x_0$ y la red debe separar una perturbación diminuta de una imagen desconocida; la tarea es difícil y se concentra en detalles muy pequeños. En el intervalo intermedio, señal y ruido son comparables y la red debe usar la estructura de la imagen para separarlos.

Con muestreo uniforme de t, el calendario lineal dedica 64.7 % de los pasos al régimen intermedio, mientras que el coseno truncado dedica 88.6 %. Por este criterio, el segundo calendario usa de forma más eficiente las iteraciones de entrenamiento.

(c) Varianza global

Por independencia de x_0 y ϵ,

$$\mathrm{Var}(x_t) = \bar{\alpha}_t \,\mathrm{Var}(x_0) + (1 - \bar{\alpha}_t)\,\mathrm{Var}(\epsilon) \tag{18}$$

$$+ 2\sqrt{\bar{\alpha}_t(1 - \bar{\alpha}_t)}\,\mathrm{Cov}(x_0, \epsilon) \tag{19}$$

$$= \bar{\alpha}_t \,\mathrm{Var}(x_0) + (1 - \bar{\alpha}_t). \tag{20}$$

La varianza medida sobre las 55000 imágenes de entrenamiento en $[-1, 1]$ fue 0.498259.

Cuadro 8: Verificación numérica de la varianza global de x_t.

t	$\bar{\alpha}_t$	Var. empírica	Var. predicha	Diferencia
1	0.999900	0.498304	0.498309	0.000005
250	0.524085	0.737226	0.737045	0.000182
500	0.078587	0.960343	0.960570	0.000227
1000	0.000040	1.000391	0.999980	0.000411

En $t = T$ la varianza predicha es 0.999980 y la media queda cerca de cero porque $\sqrt{\bar{\alpha}_T}$ es muy pequeña. Esto hace compatible el estado final del proceso forward con el inicio del proceso generativo, $x_T \sim \mathcal{N}(0, I)$.

5. Task 3 – Difusión condicional

5.1. Task 3.1 – U-Net y condicionamiento

La red recibe x_t, el instante t y una clase $c \in \{0, \ldots, 9, 10\}$, donde 10 representa la condición nula. El instante se codifica con un embedding sinusoidal:

$$e_{2i}(t) = \sin\left(t\, 10000^{-i/d}\right), \qquad e_{2i+1}(t) = \cos\left(t\, 10000^{-i/d}\right). \tag{21}$$

Este vector pasa por un MLP y se suma al embedding aprendido de clase. La condición resultante se proyecta y se suma, mediante broadcasting, en cada bloque convolucional.

Cuadro 9: Arquitectura de la U-Net condicional.

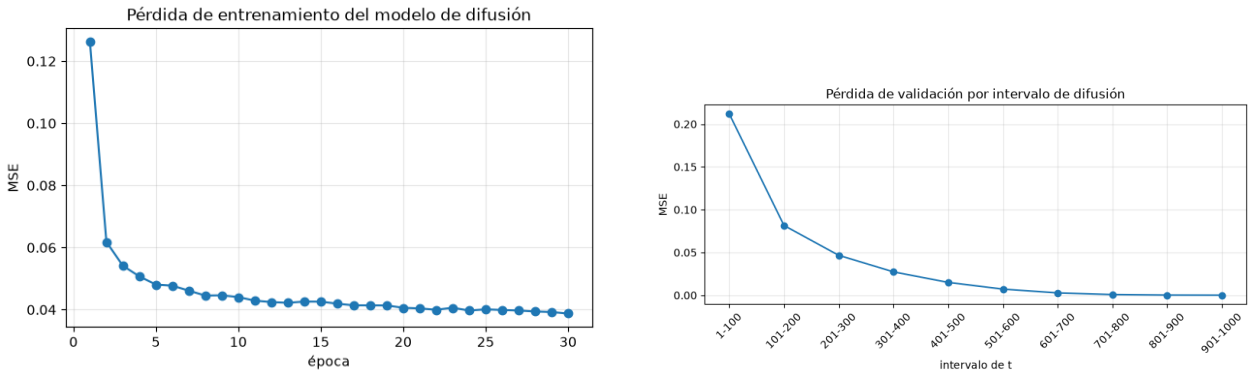
Etapa	Resolución	Canales de entrada	Canales de salida
Bloque encoder	28×28	1	32
Downsampling	14×14	32	64
Bloque encoder	14×14	64	64
Cuello de botella	14×14	64	64
Bloque decoder con skip	14×14	128	64
Upsampling	28×28	64	32
Bloque decoder con skip	28×28	64	32
Convolución final	28×28	32	1

La red tiene 373889 parámetros entrenables y devuelve tensores con la misma forma que la entrada, $(B, 1, 28, 28)$.

5.2. Task 3.2 – Entrenamiento y pérdida por intervalo

En cada iteración se muestrearon uniformemente $t \in \{1, \ldots, T\}$ y $\epsilon \sim \mathcal{N}(0, I)$. Se construyó x_t con la forma cerrada y se reemplazó la clase por la condición nula con probabilidad 0.1. La pérdida fue

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{x_0,t,\epsilon,c}\left[\|\epsilon - \epsilon_\theta(x_t, t, c)\|_2^2\right]. \tag{22}$$



(a) Pérdida de entrenamiento.

(b) Pérdida de validación por intervalo.

Figura 9: Comportamiento de la pérdida del modelo de difusión.

La pérdida de entrenamiento bajó de 0.12612 a 0.03875 en 30 épocas. El tiempo total fue 10.90 minutos. En validación, la pérdida fue máxima en $t = 1$–100 (0.21196) y mínima en $t = 901$–1000 (0.00011). Cuando la SNR es muy baja, x_t ya se parece al ruido que debe predecirse y la tarea se vuelve casi identidad. En los primeros pasos la imagen domina y separar una perturbación pequeña es más difícil. Este patrón coincide con el análisis de regímenes de la Task 2.5(b).

5.3. Task 3.3 – Muestreo ancestral

Partiendo de $x_T \sim \mathcal{N}(0, I)$, se aplicó para $t = T, \ldots, 1$:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \hat{\epsilon} \right) + \sigma_t z, \qquad \sigma_t^2 = \beta_t, \tag{23}$$

con $z = 0$ en el último paso. Para 500 imágenes con $w = 1$, el muestreo tomó 40.93 s.



Figura 10: Diez imágenes por clase generadas con muestreo ancestral y $w = 1$.

5.4. Task 3.4 – Guía sin clasificador

La predicción guiada se implementó con la convención en la que $w = 1$ equivale al modelo condicional puro:

$$\tilde{\epsilon} = \epsilon_\theta(x_t, t, \varnothing) + w\left[\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \varnothing)\right]. \tag{24}$$

El clasificador auxiliar alcanzó 88.26 % de exactitud en validación después de dos épocas. La diversidad real de referencia, medida con 50 imágenes por clase, fue 17.6752.

Cuadro 10: Fidelidad, diversidad y tiempo de generación para distintos valores de guía.

w	Fidelidad global	Diversidad global	Segundos por 100 imágenes
1	0.838	14.9273	8.1868
3	0.976	13.1091	16.3164
7	0.980	12.0806	16.5672

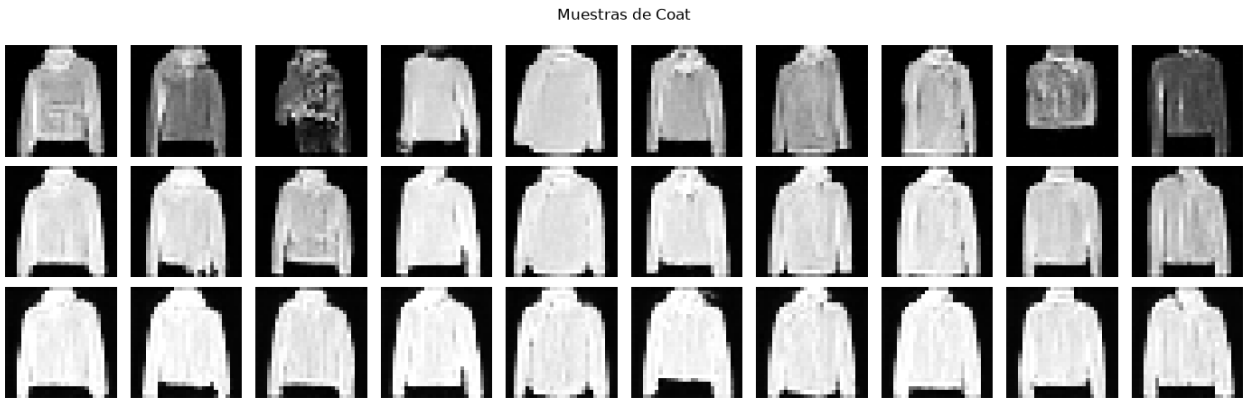


Figura 11: Muestras de la clase Coat para $w \in \{1, 3, 7\}$.

5.5. Task 3.5 – Análisis de la guía

(a) Fidelidad y diversidad

Al aumentar w de 1 a 7, la fidelidad subió de 0.838 a 0.980 y la diversidad bajó de 14.927 a 12.081. El mayor salto de fidelidad ocurrió entre $w = 1$ y $w = 3$; pasar a $w = 7$ agregó solo 0.004. En contraste, la diversidad continuó cayendo. La clase Coat fue la más afectada: su diversidad pasó de 16.710 a 9.887. La Figura 11 muestra que, con guía alta, varias muestras repiten una silueta dominante de abrigo. La condición favorece prototipos fáciles de clasificar y descarta variaciones menos típicas.

(b) Matriz de confusión

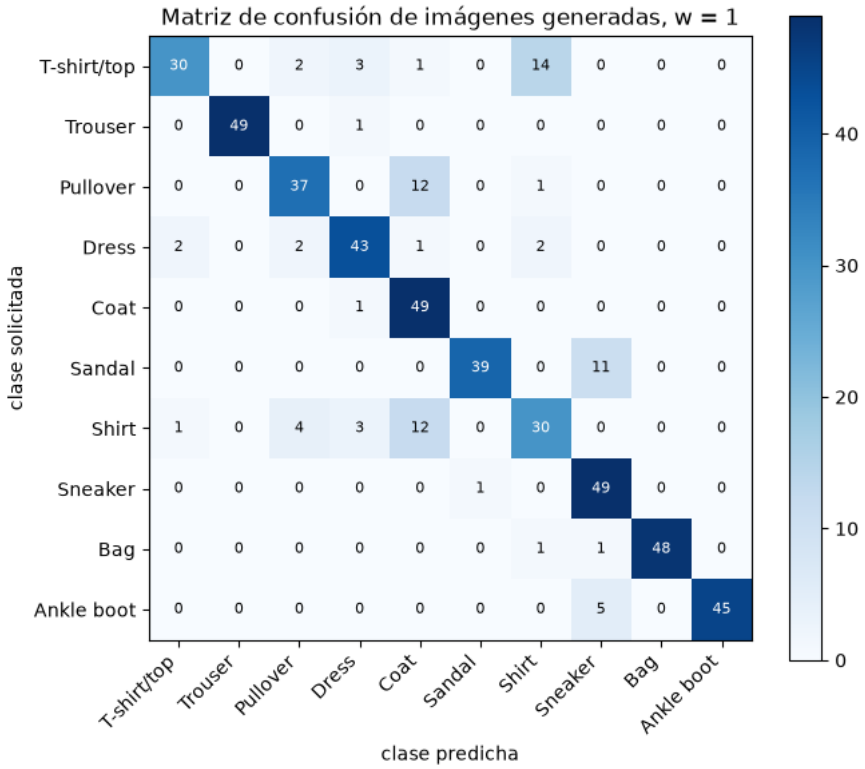


Figura 12: Matriz de confusión para imágenes generadas con $w = 1$.

El par más confundido fue T-shirt/top y Shirt, con 15 errores sumando ambas direcciones. La cuadrícula de $w = 1$ contiene prendas con mangas y siluetas parecidas en ambas filas. Parte de la confusión corresponde al generador, que produce casos ambiguos; otra parte proviene de la similitud intrínseca entre estas clases y de que el clasificador alcanza 88.26 %, no 100 %, sobre imágenes reales. Por ello no es correcto atribuir todos los errores a un solo componente.

(c) Distribución inducida por la guía

Por la regla de Bayes aplicada a gradientes,

$$\nabla_x \log p_t(x \mid c) = \nabla_x \log p_t(x) + \nabla_x \log p_t(c \mid x). \tag{25}$$

Como el score satisface $s_\theta \approx -\epsilon_\theta / \sqrt{1 - \bar{\alpha}_t}$, la misma combinación lineal se transfiere de las predicciones de ruido a los scores:

$$\begin{aligned} \tilde{s} &= s(x, t, \varnothing) + w[s(x, t, c) - s(x, t, \varnothing)] && (26)\\ &= \nabla_x \log p_t(x) + w \nabla_x \log p_t(c \mid x) && (27)\\ &= \nabla_x \log \left[p_t(x) p_t(c \mid x)^w \right]. && (28) \end{aligned}$$

Por tanto,

$$\tilde{p}_t(x \mid c) \propto p_t(x) p_t(c \mid x)^w. \tag{29}$$

Cuando w aumenta, la densidad se concentra en imágenes que el modelo considera altamente representativas de la clase. Esto explica simultáneamente el aumento de fidelidad y la pérdida de diversidad.

(d) Pérdida por intervalo

El intervalo 1–100 presentó la mayor pérdida (0.2120) y 901–1000 la menor (0.0001). El resultado coincide con la predicción de la Task 2.5(b): en SNR alto el ruido pequeño es difícil de separar de la imagen, mientras que en SNR muy bajo la entrada se aproxima al mismo ruido que debe estimarse.

6. Task 4 – Comparaciones y pass@k

6.1. Task 4.1 – VAE frente a difusión

Cuadro 11: Nitidez sobre 500 imágenes de cada origen.

Origen	Nitidez
Imágenes reales	0.0886
VAE, $\beta = 1$	0.0499
Difusión, $w = 1$	0.0736

Cuadro 12: Costo de generar 100 imágenes.

Modelo	Tiempo	Evaluaciones de red
VAE	0.000888 s	1 del decoder
Difusión	12.853 s	1000 de la U-Net

La difusión conservó 83.1 % de la nitidez real, mientras que el VAE conservó 56.3 %. Sin embargo, en esta medición la difusión fue 14470.9 veces más lenta. El VAE se ubica en el extremo de generación rápida y entrenamiento estable, a cambio de imágenes suavizadas por la pérdida cuadrática. La difusión también se entrena mediante una regresión estable y recupera mejor los bordes, pero el proceso inverso secuencial domina su costo. Para reducirlo se podría difundir en el espacio latente de un autoencoder y usar un muestreador acelerado con menos pasos, validando que la reducción no degrade la fidelidad ni la nitidez.

6.2. Task 4.2 – Estimador pass@k

Para n soluciones y c aciertos, el estimador insesgado es

$$\text{pass@}\, k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}. \tag{30}$$

Cuadro 13: Valores de pass@k y comparación con el estimador ingenuo para $k = 5$.

Problema	c	pass@1	pass@5	pass@10	Ingenuo@5
P1	0	0.000000	0.000000	0.000000	0.000000
P2	2	0.100000	0.447368	0.763158	0.409510
P3	5	0.250000	0.806308	0.983746	0.762695
P4	9	0.450000	0.970201	0.999940	0.949672
P5	14	0.700000	0.999613	1.000000	0.997570
P6	20	1.000000	1.000000	1.000000	1.000000
Promedio	–	0.416667	0.703915	0.791141	–

(a) Ejemplo P2 con $k = 5$

$$1 - \frac{\binom{18}{5}}{\binom{20}{5}} = 1 - \frac{8568}{15504} = 0.447368. \tag{31}$$

(b) Menor k para superar 0.9 en P2

Para P2 hay 18 fallos y 2 aciertos. Al cancelar factoriales,

$$\frac{\binom{18}{k}}{\binom{20}{k}} = \frac{(20-k)(19-k)}{20 \cdot 19}. \tag{32}$$

Entonces,

$$\text{pass@}\, k = 1 - \frac{(20-k)(19-k)}{380}. \tag{33}$$

Con $k = 13$ se obtiene 0.8895 y con $k = 14$ se obtiene 0.9211. El menor valor que supera 0.9 es $k = 14$.

(c) Comparación con el estimador ingenuo

Para $i \geq 0$,

$$\frac{n-c-i}{n-i} \leq \frac{n-c}{n}. \tag{34}$$

Por tanto, la probabilidad de escoger k fallos sin reemplazo satisface

$$\prod_{i=0}^{k-1} \frac{n-c-i}{n-i} \leq \left(\frac{n-c}{n}\right)^k. \tag{35}$$

Al restar ambos lados de uno, el estimador insesgado es mayor o igual que el ingenuo $1 - (1 - c/n)^k$. Si hay menos de k fallos disponibles, cualquier subconjunto de tamaño k contiene por lo menos un acierto y el estimador insesgado vale 1.

(d) Interpretación con diez intentos

P1 no se resuelve aunque se hagan diez intentos porque ninguna de las 20 soluciones originales fue correcta. P5 y P6 alcanzan pass@10 igual a 1; P3 y P4 quedan muy cerca de 1. P2 llega a 0.763, por lo que todavía falla aproximadamente una de cada cuatro veces. El promedio aumenta de 0.417 en pass@1 a 0.791 en pass@10. El modelo se beneficia mucho de generar varias propuestas y usar un verificador perfecto, pero los reintentos no ayudan en problemas para los que nunca aparece una solución correcta.

7. Conclusiones

Los experimentos reproducen tres compromisos centrales de los modelos generativos. Primero, la regularización del VAE ordena el espacio latente, pero reduce su capacidad informativa y favorece reconstrucciones suaves. Segundo, la difusión recupera imágenes más nítidas mediante una secuencia

larga de correcciones, con un costo de generación muy superior. Tercero, la guía sin clasificador mejora la correspondencia con la clase hasta cierto punto, pero concentra la distribución y reduce la diversidad.

Los controles numéricos respaldan la implementación. La KL cerrada coincidió con Monte Carlo dentro de 1.60 errores estándar; la media y la varianza del proceso forward coincidieron con sus predicciones; la aproximación analítica de $\bar{\alpha}_T$ tuvo un error relativo de solo 7.12×10^{-4}; y la varianza final fue prácticamente uno. Estos resultados indican que las fórmulas implementadas y los experimentos son coherentes entre sí.

Uso de herramientas de IA

Se utilizó una herramienta de IA como apoyo para verificar que las fórmulas se mostraran correctamente, revisar la coherencia entre el enunciado, el código y los resultados, señalar partes incompletas y ayudar a estructurar este documento a partir de las métricas y figuras ejecutadas en el notebook. El prompt principal fue: "Ayúdame a verificar que las fórmulas se vean bien, revisar que el código cumpla el enunciado y señalar lo que falte". El mismo prompt se aplicó como revisión general a las Tasks 1–4. Funcionó porque dirigió la revisión hacia requisitos verificables; los valores finales se comprobaron ejecutando el notebook completo.

Referencias

[1] D. P. Kingma e M. Welling. *Auto-Encoding Variational Bayes*. International Conference on Learning Representations, 2014.

[2] J. Ho, A. Jain y P. Abbeel. *Denoising Diffusion Probabilistic Models*. Advances in Neural Information Processing Systems, 2020.

[3] A. Q. Nichol y P. Dhariwal. *Improved Denoising Diffusion Probabilistic Models*. International Conference on Machine Learning, 2021.

[4] H. Xiao, K. Rasul y R. Vollgraf. *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*. 2017.

[5] M. Chen et al. *Evaluating Large Language Models Trained on Code*. 2021.

[6] CC3045 – Deep Learning. *Semana 14: Inteligencia artificial generativa*. Universidad del Valle de Guatemala, material de clase.